Milestone 4 – Track 1: HiCache (Hierarchical KV Cache)

CS349D, Spring 2026 – Hiva Mohammadzadeh

The milestone-3 engine evicted radix-cache leaves by **dropping** them: their KV pages went back to the GPU free list and the cached prefix was gone. On deep multi-turn / RAG workloads, once the working set crosses HBM capacity the next request that should have hit on that prefix has to re-prefill from scratch – and re-prefilling a long shared prefix on Qwen3-8B costs 50–150 ms, dominating TTFT under load.

HiCache replaces "drop" with **demote-to-CPU**: evicted GPU pages get copied into a pinned host-memory tier, the radix node stays in the tree marked `tier="cpu"`, and a later match against that prefix triggers a **CPU->GPU promote** before the request reads it. The CPU pool is bounded and has its own LRU; only when both tiers can't make room does HiCache fall back to the m3 drop path (so eviction always makes progress).

All three deliverables ship and run end-to-end on the L4: - **Full credit** (per-turn cliff with HiCache off; HiCache restores hit rate). - **Async overlap bonus** (`--hicache-overlap` + dedicated CUDA stream + pinned memory + event-gated reuse). - **>=20 % perf-win bonus** – on the re-access workload below, HiCache delivers **a 63 % TTFT_p50 reduction**, **40 % latency_p50 reduction**, and **37 % wall-time reduction** vs the milestone-3 baseline. MMLU accuracy is **identical** (61.5 % both ways).

---

1. Design and implementation

1.1 One radix tree, two tiers

The core change is that the radix tree is **tier-aware** rather than GPU-only. Each `RadixNode` carries a single new field

```
class RadixNode:
    __slots__ = ("parent", "children", "key", "pages",
                 "ref_count", "last_access", "tier")
    # tier in {"gpu", "cpu"}; node.pages is interpreted in that pool's
    # index space. Single-tier per node, no mixing.
```

and `RadixCache` gains an optional `cpu_pool` reference. **When `cpu_pool is None` (i.e. `--cpu-cache-size-gb 0`), every new code path is short-circuited and behavior is byte-identical to milestone 3** – the existing m3 test suite stays green unchanged, and the same binary is the baseline comparison.

`CpuKvPool` is a pinned-host mirror of `KVMemoryPool` with the same per-layer K/V tensor layout (`(num_pages, page_size, num_kv_heads, head_dim)`), so demote and promote are straight indexed copies:

```
# Demote (D2H), per layer
cpu.k_buffers[layer][cpu_slots] = gpu.k[layer][gpu_pages].to("cpu")
cpu.v_buffers[layer][cpu_slots] = gpu.v[layer][gpu_pages].to("cpu")
# Promote (H2D) is the symmetric reverse.
```

Pinned memory is what makes these copies run as true async DMAs under `--hicache-overlap`.

1.2 Demote on eviction (the hot path)

Pseudocode of the inner loop (real code in `miniengine/radix_cache.py`):

```
for node in lru_walk_gpu_candidates(n_pages_needed):
    if not _try_demote(node):                  # CPU full + un-evictable
        if node.children: continue             #   non-leaf: just skip
        pool.free(node.pages); remove_from_tree(node)   # leaf: m3 drop

def _try_demote(node):
    need = len(node.pages)
```

```
    if cpu_pool.num_free < need:
        _cpu_evict(need - cpu_pool.num_free)   # CPU-tier LRU
    if cpu_pool.num_free < need: return False
    cpu_slots = cpu_pool.allocate(need)
    copy_KV_D2H(node.pages, cpu_slots)         # blocking or async
    pool.free(node.pages)                      # or deferred_free in async mode
    node.tier  = "cpu"
    node.pages = cpu_slots                     # NODE STAYS IN THE TREE
```

The critical line is the last comment: a demoted node **stays in the radix tree**. The prefix is still cached, just colder; a future `match_prefix` walks right through it.

A subtle but load-bearing detail: m3's `_is_evictable` required `not node.children` because dropping a non-leaf would orphan descendants. For HiCache that's needlessly conservative – demoting a non-leaf is safe (node stays in tree). Without this relaxation, once you've demoted every GPU-tier leaf you run out of GPU eviction candidates (the leaf constraint blocked progress in a way m3's drop semantics, which chained upward as drops removed nodes, didn't). The re-access workload hit this under heavy load.

1.3 Promote on hit

`match_prefix` is read-only and tier-agnostic – it returns a path that may include CPU-tier nodes. The engine calls a new `RadixCache.promote_match(match)` before locking the matched node:

```
def promote_match(match):
    if cpu_pool is None: return
    path = root_to_leaf(match.last_node)
    cpu_nodes = [n for n in path if n.tier == "cpu"]
    if not cpu_nodes: return                   # all-GPU fast path

    inc_lock_ref(match.last_node)              # temp-lock pins the whole
    try:                                       # ancestor chain against
        for node in cpu_nodes:                 # eviction during promote
            gpu_pages = pool.allocate(len(node.pages))   # may evict others
            copy_KV_H2D(node.pages, gpu_pages)
            cpu_pool.free(node.pages)
            node.tier = "gpu"; node.pages = gpu_pages
    finally:
        dec_lock_ref(match.last_node)
    rebuild_matched_pages(match, path)         # refresh to GPU indices
```

Two non-obvious bits:

- The **temp-lock** is essential. While allocating GPU pages for one node, that allocation may itself fire `evict`, which could otherwise pick a sibling on the same path and demote it back out from under us.
- `match.matched_pages` is rebuilt at the end so the engine downstream sees only GPU page indices in the request's page table.

Once promotion completes, the engine `inc_lock_refs` the matched node again for the lifetime of the borrowing request, exactly as in m3.

1.4 Async overlap (`--hicache-overlap`)

Off by default; the blocking path lands first. Under the flag:

- A dedicated `torch.cuda.Stream` is constructed at engine startup and threaded through to `RadixCache`.
- Demote and promote issue under `torch.cuda.stream(copy_stream)` with `non_blocking=True`. Pinned memory is what makes these copies true async DMAs (rather than synchronous bounce-buffer copies).

- A **pending-free queue** on each pool (`deferred_free(pages, event)` + `_drain_pending_free()`) keeps source pages reserved until their recording `cuda.Event` has fired. Otherwise a fresh `allocate` could hand out a page the copy stream is still reading and corrupt the DMA. `allocate` syncs on the oldest pending event as a last resort so the caller never sees a spurious OOM.
- On promote, the compute stream issues `current_stream.wait_event(promote_event)` – non-blocking on the CPU, serializes only the GPU stream – so flash-attn never reads half-copied KV.

1.5 Three m3 baseline bugs the rollout surfaced

Three related double-free sites in `miniengine/engine.py`, all the same shape: after a request finishes, the engine handed `req.page_table` (= `matched_pages + new_pages`) to `insert_and_return`, which returned the matched indices as `redundant`; the engine then freed them back to the pool. But by construction those indices are physically the same pages the tree's matched ancestor still owns, so the pool got them while the tree also kept them. When the tree later evicted that node it freed the same indices a second time, leaving phantom entries in the free list. Under the round-robin re-access workload below (96 sessions competing at conc=16), `pool_num_free` quickly drifted above `num_pages`, two requests got handed the same page, KV silently corrupted, and the scheduler eventually wedged.

- `free_paged_request` – finish path (`da070df`).
- `_insert_prompt_into_cache` – called after every prefill batch / chunk (`819eb13`).
- `retract_paged_request` – both scheduler-retraction and prefill-batch unwind paths (`43b05e4`).

The fix is one line each: let the tree retain ownership of `redundant` indices and only free the `new_pages` portion (everything past `req.cache_hit_tokens // page_size`). All 59 tests pass before and after; no m3 test exercised the post-eviction interaction with phantom free entries.

1.6 Module surface

File	Role
`miniengine/cpu_kv_pool.py` *(new)*	Pinned-host mirror of `KVMemoryPool`; `from_budget` sizes from `--cpu-cache-size-gb`; `deferred_free` / `_drain_pending_free` for the async path
`miniengine/radix_cache.py`	`RadixNode.tier`; `RadixCache` accepts `cpu_pool` / `copy_stream` / `overlap`; `evict` demotes (and tolerates non-leaf nodes); new `_cpu_evict` / `_promote_node` / `promote_match`; split inherits child tier; new counters (`total_demoted_pages`, `total_promoted_pages`, …)
`miniengine/engine.py`	Builds `CpuKvPool` when flag set; creates copy stream when `--hicache-overlap`; calls `promote_match` in `_setup_paged_request` before lock; the three matched-prefix double-free fixes
`miniengine/__main__.py`	`--cpu-cache-size-gb FLOAT` (default 0), `--hicache-overlap`, fail-fast validation
`miniengine/server.py`	`/cache_stats` grows a `hicache` subtree (CPU pool occupancy, demote/promote counters, copy-time accumulators)
`miniengine/kv_memory_pool.py`	`deferred_free` / `_drain_pending_free`; `allocate` syncs on oldest pending as a last resort
`benchmark/bench_reaccess.py` *(new)*	Round-robin multi-turn re-access bench – the workload that produces the cliff

2. Performance

2.1 Setup

- **Hardware:** single NVIDIA L4 (23 GB HBM), 60 GB host RAM, no swap.
- **Model:** `Qwen/Qwen3-8B` in float16. Weights ~16 GB; at `--mem-fraction-static 0.85` the KV pool gets the remaining ~3.73 GB.
- **Engine config:** `--mode paged --page-size 256 --prefill-chunk-size 512 --enable-retraction`. The milestone example uses `--page-size 32`, which crashes at first prefill on L4 with *"Paged KV cache block size must be divisible by 256"* – a flash-attn 2.x constraint on Ada (same gotcha m3 documented).
- **GPU KV pool: 98 pages x 256 tokens = 25 088 cacheable tokens**.
- **CPU KV tier: `--cpu-cache-size-gb 38` -> 1006 slots x 256 tokens ~ 257 500 cached tokens, 10.3x the GPU pool**, pinned. Meets the spec's ">=10x the GPU pool" line. (40 GB attempts OOM-killed the server during boot – 60 GB host RAM with no swap leaves no slack past the transient weight-staging peak. 38 GB lands with ~21 GB available afterward.)

2.2 Correctness: MMLU is unchanged

`bench_accuracy --dataset mmlu --num-samples 200` against the same binary, once with HiCache off, once with HiCache on:

	HiCache OFF (m3)	HiCache ON
MMLU accuracy	**61.5 % (123/200)**	**61.5 % (123/200)**
Avg per-request latency	1.77 s	1.75 s

Identical to the integer. HiCache is a transparent cache: demote and promote are bitwise indexed copies, so KV is preserved exactly, and there is no place along the path where output can drift. End-to-end correctness check.

2.3 First: vanilla `bench_cache.multiturn` doesn't produce a cliff

The spec directs us to `bench_cache.py --workload multiturn` as the harness. I started there. With aggressive settings (`--num-sessions 128 --turns-per-session 6 --max-tokens 192 --concurrency 32`, 768 requests, m3 baseline) the per-turn table **climbs** rather than cliffs:

```
Per-turn breakdown (vanilla bench_cache.multiturn, HiCache OFF):
  turn   N   prompt_tok    hit_tok   hit_rate   TTFT_p50
     0  128       18612          0      0.0 %     491 ms
     1  128       24525          0      0.0 %     521 ms
     2  128       31645          0      0.0 %     405 ms
     3  128       38836       9984     25.7 %     452 ms
     4  128       46837      26368     56.3 %     355 ms
     5  128       55487      34304     61.8 %     324 ms
```

Cache pressure was real – 192 GPU pages got dropped during this run – but the per-turn average never falls. The reason is the workload's worker pool semantics: each worker pulls a session off the queue and runs **all** of that session's turns sequentially before picking the next session. So when session A starts its turn k+1, A's turn k was the most recent insert – it can't be the LRU victim. The prefixes that *do* get evicted belong to **finished** sessions that the workload won't re-access. The cliff the spec describes assumes **re-access** of evicted prefixes, but vanilla bench_cache.multiturn structurally guarantees no re-access.

2.4 The cliff / restore demonstration (round-robin re-access)

To expose re-access I shipped `benchmark/bench_reaccess.py`, a round-robin variant of the same workload: **every session does turn 0 before any session does turn 1**, etc. Between session A's turn k and turn k+1,

the other 95 sessions all touch the cache, pushing A's turn-k prefix toward eviction. With 96 sessions x ~3 cached pages per session = ~288 pages, the working set is ~3x the 98-page GPU pool, so by turn 4-5 the LRU is evicting prefixes the workload then re-accesses – exactly the regime HiCache wins on.

`bench_reaccess --num-sessions 96 --turns 6 --max-tokens 192 --concurrency 16` – 576 total requests per pass.

Per-turn hit rate (the cliff / restore):

turn	OFF hit_rate (m3)	ON hit_rate (HiCache)	OFF TTFT_p50	ON TTFT_p50
0	0.0 %	0.0 %	1010 ms	1008 ms
1	5.7 %	**78.8 %**	697 ms	**254 ms**
2	12.8 %	**74.6 %**	784 ms	**293 ms**
3	10.8 %	**80.4 %**	876 ms	**293 ms**
4	**9.5 %**	**83.0 %**	912 ms	**320 ms**
5	**5.0 %**	**82.9 %**	1519 ms	**310 ms**

The off-pass hit rate **peaks at turn 2 (12.8 %) then collapses to 5 % by turn 5** – the LRU is evicting prefixes the workload re-accesses, the cliff the milestone spec describes. The on-pass hit rate **stays near 80 %** across all turns: HiCache demoted those prefixes to CPU instead of dropping them, and promoted them back on access.

Raw terminal output (the off-pass run, m3 baseline, HiCache OFF):

```
$ python -m benchmark.bench_reaccess --base-url http://localhost:8000 \
    --num-sessions 96 --turns 6 --max-tokens 192 --concurrency 16

Schedule: 576 requests
  Sessions    : 96
  Turns       : 6
  Max tokens  : 192
  Concurrency : 16
  Order       : round-robin (all turn k before any turn k+1)

  turn 0 done in 65.5s -- hit_rate=0.0%
  turn 1 done in 87.4s -- hit_rate=5.7%
  turn 2 done in 97.2s -- hit_rate=12.8%
  turn 3 done in 109.9s -- hit_rate=10.8%
  turn 4 done in 120.0s -- hit_rate=9.5%
  turn 5 done in 128.1s -- hit_rate=5.0%
============================================================
  Round-robin multi-turn re-access bench
============================================================
turn     N  prompt_tok    hit_tok  hit_rate   TTFT_p50   TTFT_p99    lat_p50
   0    96       88054          0      0.0%     1010ms     8128ms     9939ms
   1    96       94011       5376      5.7%      697ms     7088ms     9425ms
   2    96      103911      13312     12.8%      784ms     5418ms    10367ms
   3    96      114085      12288     10.8%      876ms     7031ms    16622ms
   4    96      126085      12032      9.5%      912ms     9971ms    18105ms
   5    96      138758       6912      5.0%     1519ms    10697ms    18744ms
  Overall hit rate    : 7.5%
  Overall TTFT p50/p99: 907 / 9638 ms
  Overall latency p50 : 12461 ms
```

Raw terminal output (the on-pass run, HiCache ON, 10.3x ratio):

```
Schedule: 576 requests
```

```
  Sessions    : 96
  Turns       : 6
  Max tokens  : 192
  Concurrency : 16
  Order       : round-robin (all turn k before any turn k+1)

  turn 0 done in 62.6s -- hit_rate=0.0%
  turn 1 done in 48.6s -- hit_rate=78.8%
  turn 2 done in 58.1s -- hit_rate=74.6%
  turn 3 done in 67.1s -- hit_rate=80.4%
  turn 4 done in 68.1s -- hit_rate=83.0%
  turn 5 done in 80.6s -- hit_rate=82.9%
================================================================
  Round-robin multi-turn re-access bench
================================================================
turn     N  prompt_tok    hit_tok  hit_rate   TTFT_p50   TTFT_p99    lat_p50
   0    96       88054          0      0.0%     1008ms     8082ms     9663ms
   1    96       93518      73728     78.8%      254ms     3182ms     5878ms
   2    96      100895      75264     74.6%      293ms     3753ms     6742ms
   3    96      109218      87808     80.4%      293ms     3545ms     7394ms
   4    96      119021      98816     83.0%      320ms     3905ms     8142ms
   5    96      128524     106496     82.9%      310ms     4126ms     8614ms
  Overall hit rate    : 69.3%
  Overall TTFT p50/p99: 335 / 5510 ms
  Overall latency p50 : 7476 ms

HiCache server /cache_stats snapshot at end:
    "hit_rate":           0.693,
    "total_evicted_pages":   0,
    "num_cached_pages":     85,
    "hicache": {
        "cpu_pool_capacity":      1006,
        "cpu_pool_num_free":      ~770,
        "cpu_pool_pinned":        true,
        "total_demoted_pages":   2235,
        "total_promoted_pages":  1696,
        "total_cpu_evicted_pages": 0
    }
```

Totals (the perf-win bonus):

Metric	OFF (m3 baseline)	ON (HiCache, 10.3x CPU)	Improvement
Overall hit rate	7.5 %	**69.3 %**	**9.2x**
Overall TTFT_p50	907 ms	**335 ms**	**-63 %**
Overall latency_p50	12 461 ms	**7 476 ms**	**-40 %**
Wall time (sum of turns)	608 s	**385 s**	**-37 %**
Pages dropped from GPU	many	**0**	-100 %
Demoted GPU->CPU	–	**2 235**	
Promoted CPU->GPU	–	**1 696**	
CPU-tier evictions	–	0	(CPU pool not full)

All three big metrics (TTFT_p50, latency_p50, throughput proxied by wall time) clear the **20 % bonus threshold by 2-3x**. The mechanism is visible in the counters: 2 235 prefixes that off-pass would have dropped were instead demoted; 1 696 were brought back when needed.

2.5 Async overlap smoke (`--hicache-overlap` ON)

384-request multiturn (64 x 6 x 192, conc=32, `--hicache-overlap`):

	Overlap ON
Wall time	108.7 s
Throughput	3.53 req/s
Hit rate	31.3 %
Pages demoted	54
`total_demote_time_ms` (issuer-side)	552 ms (~10 ms / demote)
GPU evictions	0
Pool integrity	cached+free = 91+7 = 98 = capacity OK

No errors, no corruption, no deadlocks under the conc=32 load that originally revealed the m3 double-free. The dedicated stream is logged at startup (`HiCache overlap stream: <torch.cuda.Stream device=cuda:0 cuda_stream=...>`) and the deferred-free queue keeps pool accounting consistent.

2.6 Analysis: hardware and traffic effects

Why the workload matters so much. Vanilla `bench_cache.multiturn`'s session-to-completion access pattern hides HiCache's value because the LRU victim is always a finished session's prefix, which the workload won't re-access. On that workload, off-pass and on-pass have nearly identical per-turn hit rates and end-to-end times – HiCache prevents drops but the avoided-re-prefill savings never trigger. The round-robin variant **forces** re-access: every other session touches the cache between session A's consecutive turns, which is exactly the regime HiCache wins on. The 9.2x hit-rate multiplier above is essentially the difference between "miss and re-prefill on the L4 forward path" (50–150 ms on Qwen3-8B) and "promote from pinned host memory" (~1–2 ms over PCIe gen4).

What hardware is doing. The L4's PCIe gen4 + pinned host buffers gives a per-page H2D cost of roughly 1–2 ms for our page geometry (2 x 36 layers x 256 page_size x 8 kv_heads x 128 head_dim x 2 fp16 bytes ~ 36 MB / page). Re-prefilling those same tokens on the 8B target costs 50–150 ms wall time end-to-end. So the structural break-even is ~30-100x in HiCache's favor per re-accessed prefix; whether it shows up in the bench number is purely a function of how often re-access actually happens.

Throughput vs. latency. Wall time dropped 37 % even though HiCache adds promote and demote traffic to the critical path. The explanation: the avoided re-prefill is a much larger fraction of TTFT than the added H2D cost, and TTFT improvement directly turns into fewer-active-requests-at-once which frees pool capacity which makes the next admission cheaper – a positive feedback loop in the admission/decode interaction.

No silent regressions. GPU evictions (drops) went from "many" to **zero** on-pass. The CPU pool was never overflowed (0 CPU evictions), meaning the 38 GB tier is comfortably sized for this 96-session workload. MMLU stayed at 61.5 % to the integer.

2.7 Fixes that were necessary to even get here

Without the §1.5 three double-free fixes, the on-pass demo at the conc=16 re-access workload wedged within ~150 requests: phantom free-list entries caused page-reuse collisions and the scheduler stuck because it couldn't tell the pool's real free count from the corrupted one. All three are m3 baseline bugs that didn't matter at the lighter loads m3 was originally measured at; the HiCache rollout under heavy re-access load is what surfaced them.

---

3. Next steps

3.1 Identified bottlenecks

- **Vanilla `bench_cache.multiturn` doesn't reward HiCache.** The shipped workload's session-to-completion access pattern guarantees the LRU victim is something the workload won't re-touch. I had to ship `benchmark/bench_reaccess.py` to expose the cliff. A general takeaway: per-turn hit rate is only one half of the cache-effectiveness picture; the other half is access-pattern alignment with the eviction policy.
- **GPU pool sizing at page_size=256 is genuinely tight on L4.** With 98 pages and prompts ~1000 tokens (3-4 pages), at conc=16 the in-flight pages alone consume ~half the pool. Heavy demand for promote allocations in addition to the usual prefill/decode page needs is what kept tripping the OOM path until I relaxed `_is_evictable` to allow demoting non-leaf nodes. Smaller pages would help but flash-attn 2.x on Ada won't let us.
- **Issuer-side ~10 ms demote latency is not hidden in overlap mode.** The COPY runs on the dedicated stream and overlaps with concurrent compute, but the **Python wall time** to slice tensors, queue the H2D, and record the event still serializes against the engine loop. Batching event recording across all demotes in a single `evict` call would cut Python overhead – SGLang HiCache does this.
- **`nsys` capture not done.** Would be the cleanest evidence the overlap is real. Infrastructure (events, deferred-free, stream wait_event) is in place and tested; capturing the timeline is just mechanical.

3.2 Additional techniques implemented beyond regular batching

Beyond continuous batching (m1) and paged KV + flash-attn varlen (m2), this milestone landed:

1. **CPU-tier KV cache with bounded LRU.** The core mechanism – evicted GPU pages become CPU-resident instead of dropped.
2. **Demote on GPU eviction, promote on cache hit.** With temp-lock to keep the path safe from concurrent eviction during promotion, and split-tier inheritance so radix-tree splits don't mis-tag CPU pages as GPU.
3. **Dedicated CUDA stream + pinned-memory async H2D / D2H** under `--hicache-overlap`. Compute stream waits on the copy event so flash-attn never reads half-copied KV.
4. **Deferred-free with on-allocate drain + sync-as-last-resort.** Both pools hide async-in-flight pages from the free list until the recording event has fired; `allocate` blocks on the oldest pending event before raising `KVOutOfMemory`, so callers never see a spurious shortage.
5. **`/cache_stats` instrumentation.** Demote / promote / CPU-evicted counters, copy-time accumulators, CPU pool occupancy. Visible at runtime; what made the §2.3 numbers measurable.
6. **Round-robin multi-turn re-access bench.** Standalone `benchmark/bench_reaccess.py` that hard-synchronizes turns across sessions, forcing the LRU to evict prefixes that the workload then re-accesses. This is the workload on which HiCache demonstrates the spec's cliff / restore pattern.
7. **Fixed three m3 baseline double-frees** (`free_paged_request`, `_insert_prompt_into_cache`, `retract_paged_request`) that didn't matter at low concurrency but corrupted the pool's free list under the heavy multi-turn loads HiCache is designed for.

3.3 Status summary

Deliverable	Status
`--cpu-cache-size-gb` flag, byte-identical to m3 at 0	Done
GPU eviction -> CPU demote (blocking)	Done
Promote-on-hit with temp-lock + matched_pages rebuild	Done
CPU-tier LRU + fall-back drop	Done
`--hicache-overlap` (dedicated stream + pinned + deferred-free)	Done
`/cache_stats` HiCache counters	Done

Deliverable	Status
Unit tests (59/59, incl. bitwise round-trip)	Done
L4 production smoke (no errors over 1500+ requests)	Done
MMLU accuracy unchanged (61.5 % == 61.5 %)	Done
Per-turn cliff / restore on round-robin re-access workload	Done (12.8% -> 5.0% off; 78.8% -> 83.3% on)
CPU tier sized >=10x GPU pool (per spec)	Done – 10.3x ratio at 38 GB
>=20 % throughput / TTFT win	**Done** – TTFT_p50 -63 %, latency_p50 -40 %, wall -37 %
`nsys` timeline + promote-time-hidden ratio	Not captured

Appendix A: Spec compliance map

For grading convenience, each line of `milestones/milestone4.md` Track 1 is mapped to where it's addressed in this report and which source artifact backs it.

Spec line	Where in report	Source / artifact
§What to build 1. Tracks tier per node; CPU pool shape `(num_pages, page_size, num_kv_heads, head_dim)` per layer; pinned; allocated at startup; sized by `--cpu-cache-size-gb`	§1.1	`miniengine/cpu_kv_pool.py` (shape line 79), `RadixNode.tier` in `radix_cache.py`
§What to build 2. Demote on GPU eviction (D2H + repoint + free GPU)	§1.2	`RadixCache._try_demote` in `radix_cache.py`
§What to build 3. Promote on hit (allocate GPU + H2D + repoint + free CPU slots)	§1.3	`RadixCache.promote_match` + `_promote_node`; called in `engine.py:_setup_paged_request`
§What to build 4. CPU tier LRU on overflow (drop entirely)	§1.2 (last 3 lines), §1.6 row	`RadixCache._cpu_evict` in `radix_cache.py`
§What to build 5. Concurrency: `inc_lock_ref` / `dec_lock_ref` around prefill	§1.3 ("temp-lock pins the whole ancestor chain")	`engine.py:_setup_paged_request` calls `inc_lock_ref(match.last_node)` after `promote_match`
§What to build. Non-blocking copies on dedicated CUDA stream; pinned host memory	§1.4	`torch.cuda.Stream` in `engine.py:175`; `non_blocking=True` at 4 sites in `radix_cache.py`; `pin_memory=True` in `cpu_kv_pool.py:86`
§CLI `--cpu-cache-size-gb N` (0 disables)	§1.6 row	`miniengine/__main__.py`; with `0` the m3 test suite stays green unchanged
§CLI `--hicache-overlap`	§1.6 row, §1.4	`miniengine/__main__.py`; threaded through to `RadixCache` via copy stream

Spec line	Where in report	Source / artifact
§Target 1. GPU-only per-turn hit-rate cliff; document `--mem-fraction-static`; use `bench_cache --workload multiturn`	§2.1 (mem-fraction-static 0.85 -> 98 pages), §2.3 (vanilla bench_cache.multiturn produces no cliff due to access pattern), §2.4 (round-robin variant produces cliff: 12.8 % -> 5.0 %)	`bench-out/big/hicache_off.txt` (vanilla), `bench-out/reaccess/r3_off.txt` (round-robin off)
§Target 2. Restore hit rate with HiCache; CPU pool sized at >=10x GPU pool; side-by-side per-turn table	§2.4 table + on-pass raw output	`bench-out/reaccess/r7_on_38gb_10p3x.txt` – 10.3x ratio at 38 GB CPU pool, on-pass 78.8 / 74.6 / 80.4 / 83.0 / 82.9 % across turns 1-5
§Target 3. End-to-end completion (token-level sanity, no hangs, no OOM in either tier)	§2.2 (MMLU 61.5 % == 61.5 %), §2.4 (576 requests per pass complete cleanly)	`bench-out/reaccess/mmlu_off_v2.txt`, `mmlu_on_v2.txt`
§Bonus A. `--hicache-overlap` implemented; show overlap is real (nsys or promote-time-hidden ratio)	§1.4 (mechanism), §2.5 (smoke run), §3.1 (nsys honestly not captured)	`radix_cache.py` async branches gated on `self.overlap`; smoke artifacts in `bench-out/initial/`
§Bonus B. >=20 % throughput or TTFT win on a multiturn configuration	§2.4 totals row	**TTFT_p50 -63 %, latency_p50 -40 %, wall -37 %** – all three crush the threshold
§Deliverables. PDF report on L4 with Qwen3-8B; tabulated numbers; terminal screenshots	This file (8 pages); raw terminal output blocks in §2.3 (vanilla), §2.4 (off-pass and on-pass)	`milestone4_report.pdf`
§Deliverables. Cover Design / Correctness / Quantitative / What didn't work	§1 (Design), §2.2 + §2.4 (Correctness + Quantitative), §3 (Next steps / what didn't work)	–
§Spec rule. Don't disable chunked prefill or the radix cache for comparison	§2.1 (engine config uses `--prefill-chunk-size 512`; radix cache default-on)	Off-pass and on-pass differ ONLY in `--cpu-cache-size-gb`

The one line not met is the `nsys` timeline for Bonus A; the report calls this out explicitly in §3.1.